

Flag : `picoCTF{dh_s3cr3t_bd38f376}`

📄 Résumé de la faille

Un échange **Diffie-Hellman** cryptographiquement solide (nombre premier de 1048 bits) est utilisé pour établir un secret partagé. Mais le script qui chiffre le flag réduit ce secret énorme à **un seul octet** (`shared % 256`) avant de s'en servir comme clé XOR. Résultat : la clé réelle n'a que **256 valeurs possibles**, brute-forçable instantanément — peu importe la robustesse du calcul DH derrière.

Concept : un calcul cryptographique fort, mal utilisé ensuite (mauvaise dérivation de clé), réduit à néant toute sa sécurité.

🔧 Méthodo — les étapes

1. Lire l'énoncé et les données fournies

```
g = 2
p = <nombre premier de 1048 bits>
A = <clé publique d'Alice, g^a mod p>
b = <exposant privé de Bob – FUITÉ, ne devrait jamais être public>
enc = cfd6dcd0fceb9c4dbd7e0cc8cdccd8ccbe0dddb8c87d98c8889c2
```

- Reconnaissance immédiate : `g` , `p` , `A` , `b` = paramètres classiques d'un échange **Diffie-Hellman**.
- L'énoncé confirme : "one side of the exchange leaked something" → `b` (la clé privée de Bob) n'aurait jamais dû être exposée.

2. La solution "prévue" (calcul DH classique) - Avec `A` et `b` connus, calculer le secret partagé : `shared = A^b mod p` (= `pow(A, b, p)` en Python) - Utiliser ce secret pour déchiffrer `enc` .

3. Le raccourci trouvé (CyberChef Magic) - `enc` fait ~27 octets, en hexadécimal. - Utiliser l'opération "**Magic**" de CyberChef sur `enc` (après un "From Hex") : elle teste automatiquement plusieurs opérations et clés courantes. - Résultat : un **XOR avec la clé bf (1 octet)** donne directement le flag en clair, **sans avoir calculé le secret DH complet**.

4. Comprendre POURQUOI le raccourci marche (analyse du code source) - Le fichier `encryption.py` fourni révèle la ligne clé :

```
enc = bytes([x ^ (shared % 256) for x in flag])
```

- `shared` est un immense nombre (résultat du calcul DH sur 1048 bits), mais `shared % 256` ne garde que le **reste de la division par 256** → une seule valeur entre 0 et 255.
- La clé XOR réellement utilisée n'a donc que **256 possibilités** — un espace ridiculement petit à bruteforcer, quelle que soit la taille du nombre premier `p` utilisé pour le DH.

🔍 La vraie faille : mauvaise dérivation de clé

Le protocole Diffie-Hellman en lui-même n'est PAS cassé — avec un `p` de 1048 bits, personne ne peut retrouver `b` à partir de `B` par calcul direct (problème du logarithme discret, réputé difficile).

Le problème est ailleurs : comment le secret DH est transformé en clé de chiffrement. `shared % 256` écrase presque toute l'information du secret pour n'en garder qu'un octet. C'est un vrai anti-pattern de sécurité, qu'on retrouve parfois dans du code réel mal audité.

La bonne pratique aurait été de passer `shared` dans une fonction de dérivation de clé sérieuse (ex : **HKDF**, ou au minimum un hash comme SHA-256) pour en tirer une clé de taille appropriée (16/32 octets pour AES par exemple) — pas un simple modulo brutal.

📦 Outils utilisés

Outil	Rôle
CyberChef (From Hex + XOR + Magic)	Trouver/appliquer la clé XOR
Lecture du code source (<code>encryption.py</code>)	Comprendre la vraie faille, confirmer le raisonnement

✅ Réflexes à retenir

- Quand un fichier de **code source** est fourni avec un challenge crypto, c'est une mine d'or : il donne le protocole exact utilisé, pas besoin de deviner.
- CyberChef "Magic"** peut trouver des raccourcis exploitables (comme un XOR à 1 octet) même sans comprendre toute la théorie cryptographique derrière — utile pour détecter vite une faiblesse de dérivation de clé.
- Un calcul cryptographique robuste (DH avec un grand nombre premier) ne protège rien si la clé qui en est dérivée est ensuite tronquée ou mal utilisée. La sécurité se juge sur la chaîne complète, pas sur un seul maillon.
- `valeur % 256` = clé XOR d'un seul octet = 256 possibilités = cassable en brute force quasi instantané, quelle que soit la complexité du calcul qui a produit `valeur` .

🏠 Dans la vraie vie

Ce type de faille (calcul crypto fort + dérivation de clé faible) existe réellement dans du code de production mal audité. Exemples : - Utiliser directement le résultat d'un échange DH/ECDH comme clé AES sans passer par une KDF (Key Derivation Function) - Tronquer un hash ou un secret pour "gagner de la place" - Réutiliser un secret mathématique brut là où il faudrait une fonction de dérivation dédiée (HKDF, PBKDF2, Argon2...)

La leçon : la force d'un protocole cryptographique ne vaut que ce que vaut son **implémentation complète**, du calcul du secret jusqu'à son usage final.